

Workflow Persistence Layer

Workflow Persistence Layer (MongoDB)

Overview

This document defines the **Workflow Persistence Layer** for your event-driven, Redis-backed, polyglot agent system. It provides durable storage for:

- Workflow definitions
 - Workflow execution state
 - Node-level state snapshots
 - Edge execution metadata
 - Workflow lifecycle transitions
- This persistence layer integrates with:

- Gateway (Fastify)
- Agent Harness (Node/Python)
- Workflow Engine (RxJS)
- Frontend (SSE → RxJS → UI)

It ensures that workflows can:

- Pause
- Resume
- Recover from failure
- Span multiple agents
- Persist human-in-the-loop checkpoints

Folder Structure

```
/db
index.ts
models/
  WorkflowDefinition.ts
  WorkflowExecution.ts
  WorkflowNodeState.ts
  WorkflowEdgeState.ts
  WorkflowCheckpoint.ts
```

File: models/WorkflowDefinition.ts

```
import { Schema, model } from "mongoose";

const WorkflowNodeSchema = new Schema({
  id: String,
  type: String,
  sessionId: String,
  config: Schema.Types.Mixed,
  position: Schema.Types.Mixed,
  inputs: [String],
  outputs: [String]
});

const WorkflowEdgeSchema = new Schema({
  id: String,
  fromNodeId: String,
  fromPort: String,
  toNodeId: String,
  toPort: String,
  operator: String
});

const WorkflowDefinitionSchema = new Schema(
  {
    workflowId: { type: String, required: true },
    name: { type: String, required: true },
    nodes: [WorkflowNodeSchema],
    edges: [WorkflowEdgeSchema],
    createdAt: { type: Date, default: Date.now }
  },
  { timestamps: true }
);

WorkflowDefinitionSchema.index({ workflowId: 1 });

export const WorkflowDefinitionModel = model(
  "WorkflowDefinition",
  WorkflowDefinitionSchema
);
```

File: models/WorkflowExecution.ts

```
import { Schema, model } from "mongoose";

const WorkflowExecutionSchema = new Schema(
  {
    workflowId: { type: String, required: true },
    executionId: { type: String, required: true },
    state: {
      type: String,
      enum: ["idle", "running", "waiting", "paused", "completed", "failed"],
      default: "idle"
    },
    currentNodeId: { type: String },
    createdAt: { type: Date, default: Date.now },
    updatedAt: { type: Date, default: Date.now }
  },
  { timestamps: true }
);

WorkflowExecutionSchema.index({ workflowId: 1 });
WorkflowExecutionSchema.index({ executionId: 1 });

export const WorkflowExecutionModel = model(
  "WorkflowExecution",
  WorkflowExecutionSchema
);
```

File: models/WorkflowNodeState.ts

```
import { Schema, model } from "mongoose";

const WorkflowNodeStateSchema = new Schema(
  {
    executionId: { type: String, required: true },
    nodeId: { type: String, required: true },

    state: {
      type: String,
      enum: ["pending", "running", "waiting", "completed", "failed"],
      default: "pending"
    },

    lastEvent: Schema.Types.Mixed,
    updatedAt: { type: Date, default: Date.now }
  },
  { timestamps: true }
);

WorkflowNodeStateSchema.index({ executionId: 1 });
WorkflowNodeStateSchema.index({ nodeId: 1 });

export const WorkflowNodeStateModel = model(
  "WorkflowNodeState",
  WorkflowNodeStateSchema
);
```

File: models/WorkflowEdgeState.ts

```
import { Schema, model } from "mongoose";

const WorkflowEdgeStateSchema = new Schema(
  {
    executionId: { type: String, required: true },
    edgeId: { type: String, required: true },

    lastValue: Schema.Types.Mixed,
    operator: String,
    updatedAt: { type: Date, default: Date.now }
  },
  { timestamps: true }
);

WorkflowEdgeStateSchema.index({ executionId: 1 });
WorkflowEdgeStateSchema.index({ edgeId: 1 });

export const WorkflowEdgeStateModel = model(
  "WorkflowEdgeState",
  WorkflowEdgeStateSchema
);
```

File: models/WorkflowCheckpoint.ts

```
import { Schema, model } from "mongoose";

const WorkflowCheckpointSchema = new Schema(
  {
    executionId: { type: String, required: true },
    nodeId: { type: String, required: true },

    snapshot: Schema.Types.Mixed,
    createdAt: { type: Date, default: Date.now }
  },
  { timestamps: true }
);

WorkflowCheckpointSchema.index({ executionId: 1 });
WorkflowCheckpointSchema.index({ nodeId: 1 });

export const WorkflowCheckpointModel = model(
  "WorkflowCheckpoint",
  WorkflowCheckpointSchema
);
```

Integration Notes

Gateway Responsibilities

- Create workflow execution records
- Update workflow execution state
- Store node/edge state transitions
- Store checkpoints when human approval is required

Agent Harness Responsibilities

- Emit node-level events to Redis
- Save node snapshots before pausing
- Load snapshots when resuming
- Update node/edge state in MongoDB

Workflow Engine Responsibilities

- Subscribe to session events
- Update node/edge state based on RxJS pipeline
- Trigger persistence writes

Frontend Responsibilities

- Display workflow progress
- Display paused nodes
- Trigger resume actions

Summary

This Workflow Persistence Layer provides:

- Durable workflow execution tracking
- Node and edge state storage
- Checkpointing for human-in-the-loop workflows
- Snapshot storage for agent pause/resume
- Indexed schemas for fast workflow recovery

It is fully aligned with your Gateway, Agent Harness, Redis event schema, and workflow engine architecture.